

ESTRUCTURAS DE DATOS

Trabajo Práctico N° 2 - Repaso funciones y Recursión

Ejercicio 1: Definir una función que calcule la suma de todos los números enteros comprendidos entre cero y un número entero positivo dado.

Primero vamos a pensar que es lo que tiene que hacer la función; supongamos $n=10$, entonces tenemos que hacer una función que haga la sumatoria de $i=0$ hasta n ; es decir, en este caso, $0+1+2+3+4+5+6+7+8+9+10$. Veamos de forma iterativa:

```
def suma_numeros_enteros(n):  
    suma = 0  
    for i in range(n+1):  
        suma += i  
    return suma
```

Y ahora, a lo que nos interesa, de forma recursiva:

```
def suma_numeros_enteros_recursiva(n):  
    if n == 0:  
        return 0  
    else:  
        return n + suma_numeros_enteros_recursiva(n-1)
```

Ejercicio 2: Implementar una función que me permita ver si un número es capicua o no.

Nuevamente, PENSEMOS que es lo que tiene que resolver la función, dado un numero, por ejemplo 21512, ver si es capicua o no, y para esto podemos ir comparando los extremos, si son iguales, puedo descartarlos y seguir comparando. O sea, del numero original comparamos $2=2$, como son iguales los descarto y comparo entonces $1=1$, y así sucesivamente.

De manera iterativa:

```
def es_capicua_iterativo(numero):  
    numero_str = str(numero)  
    longitud = len(numero_str)  
    for i in range(longitud // 2):  
        if numero_str[i] != numero_str[longitud - i - 1]:  
            return False  
    return True
```

La idea para cuando se hace recursivo es la misma, comparar los extremos, pero a la llamada de la función, le pasamos dicho numero SIN los extremos.

```
def es_capicua_recursivo(numero):
    numero_str = str(numero)
    longitud = len(numero_str)
    if longitud <= 1:
        return True
    elif numero_str[0] != numero_str[longitud - 1]:
        return False
    else:
        return es_capicua_recursivo(numero_str[1:longitud - 1])
```

ACLARACIÓN: siempre usamos el numero como string, para facilitar manejar la cadena.

Ejercicio 3: Dada una secuencia de caracteres, obtener dicha secuencia invertida.

De manera iterativa:

```
def invertir_secuencia_iterativa(secuencia):
    secuencia_invertida = ""
    for i in range(len(secuencia)-1, -1, -1):
        secuencia_invertida += secuencia[i]
    return secuencia_invertida
```

De manera recursiva:

```
def invertir_secuencia_recursiva(secuencia):
    if len(secuencia) <= 1:
        return secuencia
    else:
        return secuencia[-1] + invertir_secuencia_recursiva(secuencia[:-1])
```

La idea en ambas es la misma, ir manejando el indice en negativo para tomar de atrás hacia adelante.

Ejercicio 4: Definir una función recursiva que imprime los números sucesivos desde n hasta 10. Por ejemplo funcion_sucesivo (6) Imprimirá: 6,7,8,9,10

```
def funcion_sucesivo(n):
    # Verificamos si n es menor o igual a 10
    if n <= 10:
        # Imprimimos el número actual
        print(n, end=", " if n < 10 else "")
        # Llamamos a la función recursivamente con el siguiente número
        funcion_sucesivo(n + 1)
```

Ejercicio 5: Desarrollar un algoritmo que permita calcular la siguiente serie: $h(n) = 1 + 1/2 + 1/3 + \dots + 1/n$

```
def serie_iterativa(n):  
    suma = 0  
    for i in range(1, n+1):  
        suma += 1/i  
    return suma  
  
def serie_recursiva(n):  
    if n == 1:  
        return 1  
    else:  
        return 1/n + serie_recursiva(n-1)
```